

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Name:      Demarce Willaims
% Class:     Signals and Systems
% Lab #:     01
% Date:      September 4, 2025
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
A = [1 0 4 5 3 9 0 2]
```

```
A = 1×8
    1     0     4     5     3     9     0     2
```

```
a = [4 5 0 2 0 0 7 1]
```

```
a = 1×8
    4     5     0     2     0     0     7     1
```

```
B = [A a]
```

```
B = 1×16
    1     0     4     5     3     9     0     2     4     5     0     2     0 ...
```

```
C = [a,A]
```

```
C = 1×16
    4     5     0     2     0     0     7     1     1     0     4     5     3 ...
```

```
D = zeros(1,50)
```

```
D = 1×50
    0     0     0     0     0     0     0     0     0     0     0     0     0 ...
```

```
E = ones(1,100)
```

```
E = 1×100
    1     1     1     1     1     1     1     1     1     1     1     1     1 ...
```

```
F = 1:30
```

```
F = 1×30
    1     2     3     4     5     6     7     8     9    10    11    12    13 ...
```

```
G = 25:-3:1
```

```
G = 1×9
    25    22    19    16    13    10     7     4     1
```

```
H = 0:0.2:2.0
```

```
H = 1×11
    0    0.2000    0.4000    0.6000    0.8000    1.0000    1.2000    1.4000 ...
```

```
% Define the vectors
```

```
V1 = [1 2 3 4 5 6 7 8 9 0];  
V2 = [0.3 1.2 0.5 2.1 0.1 0.4 3.6 4.2 1.7 0.9];  
V3 = [4 4 4 4 3 3 2 2 2 1];
```

```
%% (a) Sum of all elements
```

```
sumV1 = sum(V1);  
sumV2 = sum(V2);  
sumV3 = sum(V3);
```

```
disp(['Sum of V1: ', num2str(sumV1)]);
```

Sum of V1: 45

```
disp(['Sum of V2: ', num2str(sumV2)]);
```

Sum of V2: 15

```
disp(['Sum of V3: ', num2str(sumV3)]);
```

Sum of V3: 29

```
%% (b) Fifth element of each vector
```

```
fifthV1 = V1(5);  
fifthV2 = V2(5);  
fifthV3 = V3(5);
```

```
disp(['5th element of V1: ', num2str(fifthV1)]);
```

5th element of V1: 5

```
disp(['5th element of V2: ', num2str(fifthV2)]);
```

5th element of V2: 0.1

```
disp(['5th element of V3: ', num2str(fifthV3)]);
```

5th element of V3: 3

```
% Trying V1(0) and V1(11)
```

```
% Uncomment the lines below to see the errors in MATLAB
```

```
% V1(0) % Error: Subscript indices must either be real positive integers or  
logicals.
```

```
% V1(11) % Error: Index exceeds the number of array elements.
```

```
%% (c) New vectors V4 and V5 from V2
```

```
V4 = V2(1:5); % First 5 elements  
V5 = V2(end-4:end); % Last 5 elements
```

```
disp('V4 (first 5 of V2):');
```

V4 (first 5 of V2):

```
disp(V4);
```

0.3000 1.2000 0.5000 2.1000 0.1000

```
disp('V5 (last 5 of V2):');
```

V5 (last 5 of V2):

```
disp(V5);
```

0.4000 3.6000 4.2000 1.7000 0.9000

```
%% (d) Vectors V6, V7, V8 from V2
```

```
V6 = V2;
```

```
V6(6) = [];                    % Remove 6th element
```

```
V7 = V2;
```

```
V7(7) = 1.4;                  % Change 7th element to 1.4
```

```
V8 = V2([1 3 5 7 9]); % Pick 1st, 3rd, 5th, 7th, 9th elements
```

```
disp('V6 (V2 with 6th element removed):');
```

V6 (V2 with 6th element removed):

```
disp(V6);
```

0.3000 1.2000 0.5000 2.1000 0.1000 3.6000 4.2000 1.7000 0.9000

```
disp('V7 (V2 with 7th element changed to 1.4):');
```

V7 (V2 with 7th element changed to 1.4):

```
disp(V7);
```

0.3000 1.2000 0.5000 2.1000 0.1000 0.4000 1.4000 4.2000 1.7000 0.9000

```
disp('V8 (1st,3rd,5th,7th,9th of V2):');
```

V8 (1st,3rd,5th,7th,9th of V2):

```
disp(V8);
```

0.3000 0.5000 0.1000 3.6000 1.7000

```
%% Additional operations
```

```
disp('9 - V1:');
```

9 - V1:

```
disp(9 - V1);
```

8 7 6 5 4 3 2 1 0 9

```
disp('V1 * 5:');
```

V1 * 5:

```
disp(V1 * 5); % Matrix multiplication if dimensions allow, but V1 is 1x10
```

5 10 15 20 25 30 35 40 45 0

```
disp('V1 + V2:');
```

V1 + V2:

```
disp(V1 + V2); % Element-wise addition
```

1.3000 3.2000 3.5000 6.1000 5.1000 6.4000 10.6000 12.2000 10.7000 0.9000

```
disp('V1 - V3:');
```

V1 - V3:

```
disp(V1 - V3); % Element-wise subtraction
```

-3 -2 -1 0 2 3 5 6 7 -1

```
disp('V1 .* V2 (element-wise multiplication):');
```

V1 .* V2 (element-wise multiplication):

```
disp(V1 .* V2);
```

0.3000 2.4000 1.5000 8.4000 0.5000 2.4000 25.2000 33.6000 15.3000 0

```
% V1 * V2: Attempt matrix multiplication
```

```
% V1 is 1x10, V2 is 1x10 – this is not valid matrix multiplication
```

```
% To fix: make one vector a column
```

```
result_matrix_mult = V1 * V2'; % 1x10 * 10x1 = scalar
```

```
disp('V1 * V2' (matrix multiplication):');
```

V1 * V2' (matrix multiplication):

```
disp(result_matrix_mult);
```

89.6000

```
disp('V1.^2 (element-wise square):');
```

V1.^2 (element-wise square):

```
disp(V1.^2);
```

1 4 9 16 25 36 49 64 81 0

```
disp('V1.^V3 (element-wise power):');
```

V1.^V3 (element-wise power):

```
disp(V1.^V3);
```

```
1    16    81   256   125   216    49    64    81     0
```

```
% V1^V3: Not valid because matrix exponentiation requires square matrix  
% Uncomment to see the error  
% V1^V3
```

```
disp('V1 == V3:');
```

```
V1 == V3:
```

```
disp(V1 == V3);
```

```
0    0    0    1    0    0    0    0    0    0
```

```
disp('V1 > 6:');
```

```
V1 > 6:
```

```
disp(V1 > 6);
```

```
0    0    0    0    0    0    1    1    1    0
```

```
disp('V1 > V3:');
```

```
V1 > V3:
```

```
disp(V1 > V3);
```

```
0    0    0    0    1    1    1    1    1    0
```

```
disp('V3 - (V1 > 2):');
```

```
V3 - (V1 > 2):
```

```
disp(V3 - (V1 > 2)); % Logical vector treated as 0 or 1
```

```
4    4    3    3    2    2    1    1    1    1
```

```
disp('(V1 > 2) & (V1 < 6):');
```

```
(V1 > 2) & (V1 < 6):
```

```
disp((V1 > 2) & (V1 < 6));
```

```
0    0    1    1    1    0    0    0    0    0
```

```
disp('(V1 > 2) | (V1 < 6):');
```

```
(V1 > 2) | (V1 < 6):
```

```
disp((V1 > 2) | (V1 < 6));
```

```
1 1 1 1 1 1 1 1 1 1
```

```
disp('any(V1):');
```

```
any(V1):
```

```
disp(any(V1)); % True if any non-zero elements exist
```

```
1
```

```
disp('all(V1):');
```

```
all(V1):
```

```
disp(all(V1)); % True if all elements are non-zero
```

```
0
```

```
%% ----- 4. Flow Control -----
```

```
%% (a) Nested FOR loops: Accumulating a value into vector A
```

```
% Initialize A as a zero vector with 5 elements
```

```
A = zeros(1, 5); % A = [0 0 0 0 0]
```

```
% Loop over n from 1 to 4
```

```
for n = 1:4
```

```
    % For each n, loop m from 1 to 3
```

```
    for m = 1:3
```

```
        % Add the product n*m to every element in A
```

```
        A = A + n * m;
```

```
    end
```

```
end
```

```
% After loops, A will have 60 added to each element
```

```
% Final A = [60 60 60 60 60]
```

```
disp('Result of nested loops A =');
```

```
Result of nested loops A =
```

```
disp(A);
```

```
60 60 60 60 60
```

```
%% (b) IF / ELSEIF / ELSE logic with logical testing
```

```
% Define a vector B with elements 1 and 0
```

```
B = [1 0];
```

```
% 'all(B)' is false (not all elements are non-zero)
```

```
% 'any(B)' is true (at least one element is non-zero)
```

```
% So the elseif branch executes, adding 2 to each element
```

```
if all(B)
```

```
    B = B + 1;
```

```
elseif any(B)
    B = B + 2;
else
    B = B + 3;
end
% Final B = [3 2]
disp('Result of conditional logic B =');
```

Result of conditional logic B =

```
disp(B);
```

```
3    2
```

```
%% (c) WHILE loop with condition checking
% Create a vector C from 7 to 21 with step of 2
C = 7:2:21; % C = [7 9 11 13 15 17 19 21]
num = 0; % Counter for number of iterations

% Subtract 3 from every element of C until at least one element <= 0
while all(C > 0)
    C = C - 3;
    num = num + 1;
end
% Loop exits after 3 iterations
% Final C = [-2 0 2 4 6 8 10 12]
% Final num = 3
disp('Result of while loop C =');
```

Result of while loop C =

```
disp(C);
```

```
-2    0    2    4    6    8   10   12
```

```
disp('Number of iterations:');
```

Number of iterations:

```
disp(num);
```

```
3
```

```
%% (d1) Loop vs Vectorization: Multiplying numbers by 5

% Using a FOR loop to create vector H1
for i = 1:20
    H1(i) = i * 5;
end

% Using vectorized code to create H2 (more efficient)
H2 = (1:20) * 5;

% Both H1 and H2 are the same
```

```
disp('H1 created using loop:');
```

H1 created using loop:

```
disp(H1);
```

5 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90 95

```
disp('H2 created using vectorization:');
```

H2 created using vectorization:

```
disp(H2);
```

5 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90 95

```
%% (d2) Loop vs Vectorization: Computing sin(n * pi / 10)
```

```
% Using loop
```

```
for n = 1:100000  
    x1(n) = sin(n * pi / 10);  
end
```

```
% Using vectorized operation
```

```
n = 1:100000;  
x2 = sin(n * pi / 10);
```

```
% They produce the same result (we won't display all for brevity)
```

```
disp('x1 and x2 produce same result (first 5 values shown):');
```

x1 and x2 produce same result (first 5 values shown):

```
disp(x1(1:5));
```

0.3090 0.5878 0.8090 0.9511 1.0000

```
disp(x2(1:5));
```

0.3090 0.5878 0.8090 0.9511 1.0000

```
%% Compare MATLAB Script and Function
```

```
% This example demonstrates the difference between a script and a function  
% using simple addition of vectors.
```

```
%% -----
```

```
% (a) Run the script: myscript.m
```

```
% Content of myscript.m:
```

```
% x = 1:5;  
% y = 6:10;  
% g = x + y;
```

```
% Run the script
```

```
%myscript
```



```

% Expectation:
% - Variables x, y, and g are created in the workspace.
% - You can access them directly:
%x      % Expected: [1 2 3 4 5]
%y      % Expected: [6 7 8 9 10]
%g      % Expected: [7 9 11 13 15]

% Try assigning the script to a variable (invalid operation)
%z = myscript
% This will cause an error:
% "Script myscript cannot be used as a function or return output arguments."

%% -----
% (b) Clear workspace
%clear
% Removes all variables. x, y, g no longer exist.

%% -----
% (c) Run the function: myfunction.m
% Content of myfunction.m:
%   function g = myfunction(x, y)
%       g = x + y;
%   end

% Define inputs
%x = 1:5;
%y = 6:10;

% Call function without inputs (invalid)
%myfunction
% This will cause an error:
% "Not enough input arguments." The function requires x and y as inputs.

% Call function with inputs and assign result to a variable
% z = myfunction(x, y)
% z should be: [7 9 11 13 15]

% Try accessing g
%g
% This will cause an error:
% "Undefined function or variable 'g'."
% g was defined inside the function and is not available globally.
% g is local to the function and does not exist in the workspace

% Try using different inputs
%a = 1:10;
%b = 2:11;
%myfunction(a, b)
% This will compute and display the result: [3 5 7 9 11 13 15 17 19 21]
% If not assigned to a variable, the result is shown but not saved.

```